

ECE 4301 Midterm 2 - True Random Number Generation on Raspberry Pi 5 for a Crypto Engine

Introduction

A True Random Number Generator (TRNG) produces unpredictable values derived from physical processes. Cryptographic systems depend on such entropy to generate keys, nonces, and salts that must not be guessable by an attacker. If the randomness source is biased or predictable, protocols such as RSA, ECDH, and AES-GCM lose their security guarantees. This project implements and tests a TRNG on the Raspberry Pi 5. The Raspberry Pi includes a hardware TRNG in its system-on-chip, which feeds entropy into the Linux kernel. A small Rust program was written to retrieve random bytes from this interface and verify the output using simple statistical checks and plots.

Types of TRNGs

Although TRNG designs vary, most practical systems fall into a few physical categories.

Thermal-Noise TRNGs

These extract entropy from the random motion of electrons inside resistors or diodes. Thermal agitation creates small voltage fluctuations known as Johnson–Nyquist noise. This noise is amplified, digitized, and post-processed to produce random bits. Thermal-noise TRNGs are simple, reliable, and widely used.

Avalanche-Noise TRNGs

A reverse-biased diode operated near breakdown produces unpredictable avalanche events through impact ionization. These current pulses contain strong noise that can be directly sampled. Avalanche-noise TRNGs provide high-bandwidth entropy and are commonly used in discrete hardware RNG circuits.

Oscillator-Jitter TRNGs

Ring oscillators and other clock circuits exhibit small timing variations due to temperature changes, voltage fluctuations, and transistor-level noise. By sampling one oscillator against

another, the system converts phase jitter into random bits. Many modern SoCs incorporate jitter-based TRNG blocks.

These three categories represent the primary physical mechanisms used in TRNG hardware. All of them rely on nondeterministic physical behavior.

Physical Principles of TRNGs

All TRNGs follow the same basic structure:

1. **Physical Entropy Source:** thermal agitation, avalanche breakdown, or oscillator jitter.
2. **Sampling:** convert analog noise or timing variation into digital bits.
3. **Debiasing:** eliminate simple patterns (e.g., XOR mixing or Von Neumann correction)
4. **Conditioning:** apply a cryptographic hash or mixer.

On Linux systems such as Raspberry Pi OS, the kernel collects hardware entropy from the SoC and from system events. The kernel's entropy pool is then used to provide random bytes through the `getrandom()` system call. This ensures that applications receive cryptographic-grade randomness without interacting with analog hardware directly.

TRNG Implementation on Raspberry Pi 5

The Raspberry Pi 5's SoC includes a hardware TRNG. The Linux kernel continuously samples this entropy source and incorporates it into its internal randomness pool.

Implementation details

A Rust program was developed to demonstrate the TRNG:

1. Request 4096 bytes from `OsRng`.
2. Attempt to read 4096 bytes from `/dev/hwrng`.
3. XOR the two buffers to conservatively mix independent entropy sources.
4. Print the final bytes in hexadecimal form.

Conclusion

This project examined the operation of TRNGs and implemented a hardware-based TRNG on the Raspberry Pi 5. Thermal-noise, avalanche-noise, and oscillator-jitter TRNGs were briefly reviewed to illustrate the common physical mechanisms behind hardware randomness. Using Rust's `OsRng` and the `/dev/hwrng` device, randomness was collected from the Pi's on-chip TRNG and analyzed with simple statistical methods. The results confirm that the device produces high-quality, unbiased entropy suitable for cryptographic use.

This TRNG supports secure key generation and nonce creation in the Raspberry Pi crypto engine and forms a foundational component in the larger encrypted-video pipeline developed for the course.